SANJIVANI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

At: kopargaon, Dist:Ahmednagar, Maharashtra, India – 423601

DEPARTMENT (Artificial Intelligence and Machine Learning)

COURSE: DevOps and Cloud (IBM)

(24UAMAE201)

Academic Year 2026-2027

Mini Project Unit 4

NAME	**Vaishnavi Jadhav**
PRN NO	**2125UMLF1100**
SEM/YEAR	**2nd Year/3rd Sem**
BATCH	**SY(A)**

SUBMITTED TO

Mr. Karan Gaikwad Sir
(Professor)

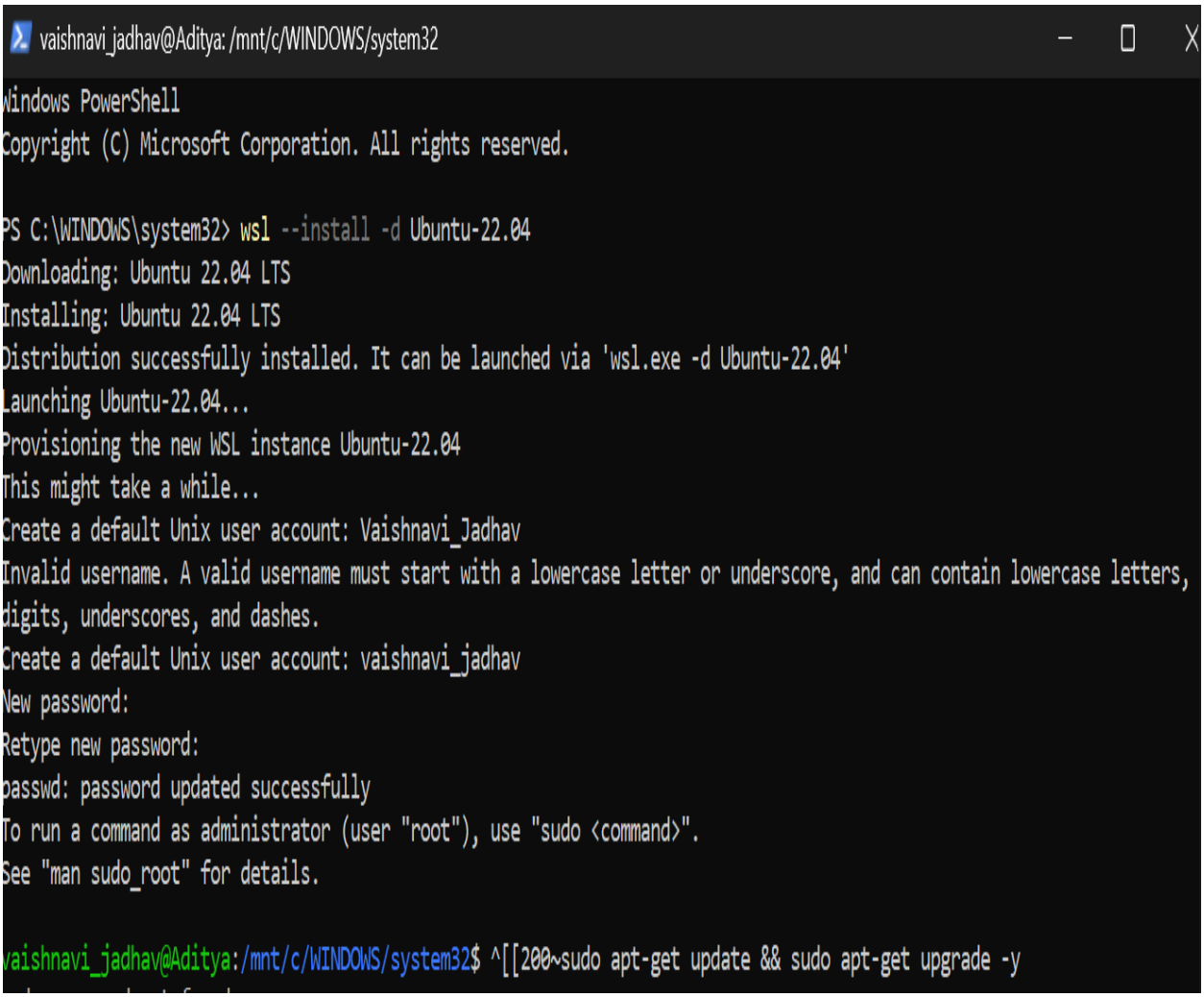
SUBMITTED BY

Vaishnavi Jadhav
(B.Tech 2nd year)

PRN: 2125UMLF1100

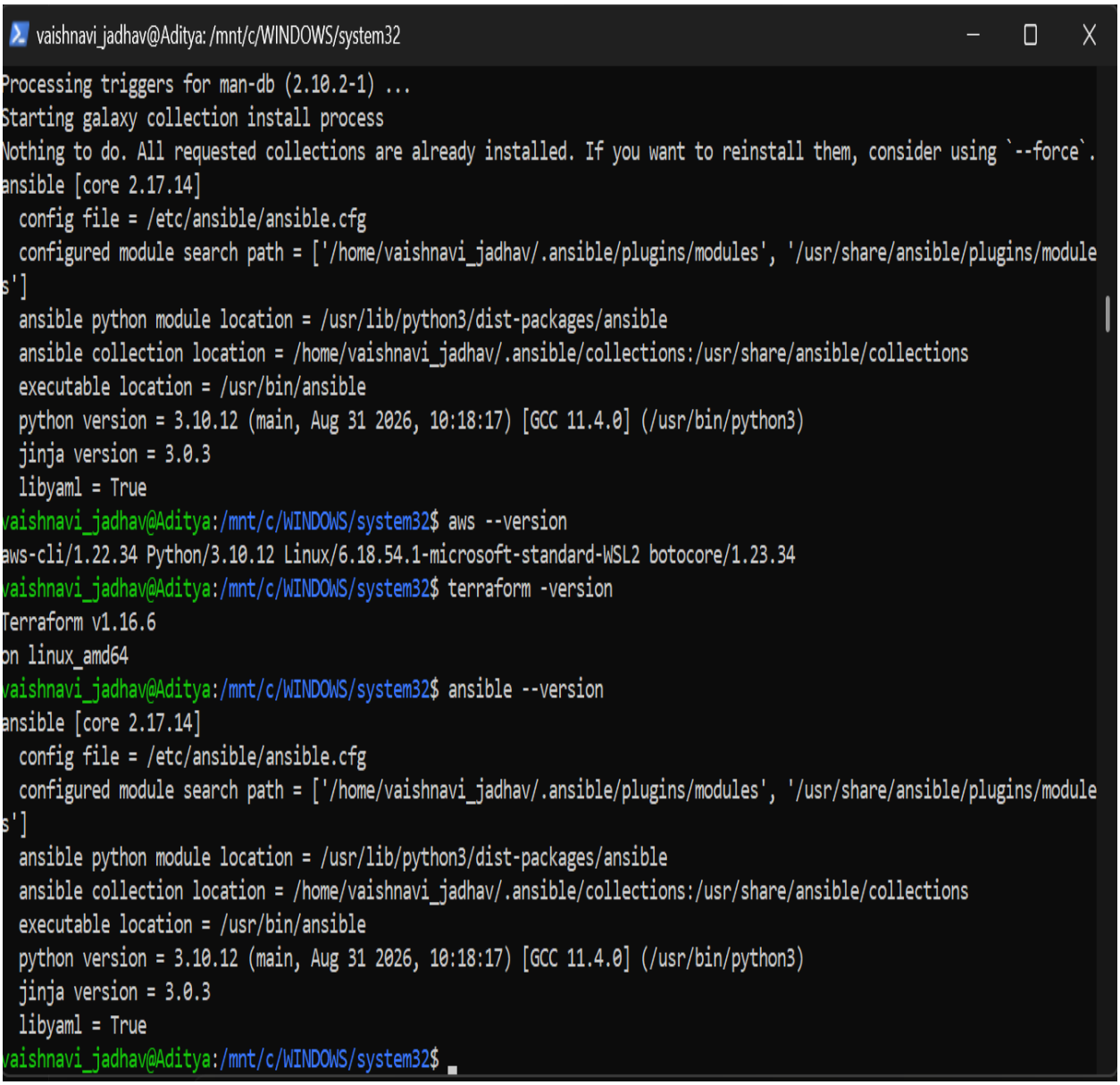
Step 0: Set up WSL2 on Windows

- Ubuntu 22.04 running under WSL2 on Windows. It is the Linux control node for Terraform and Ansible, because Ansible does not run on native Windows.



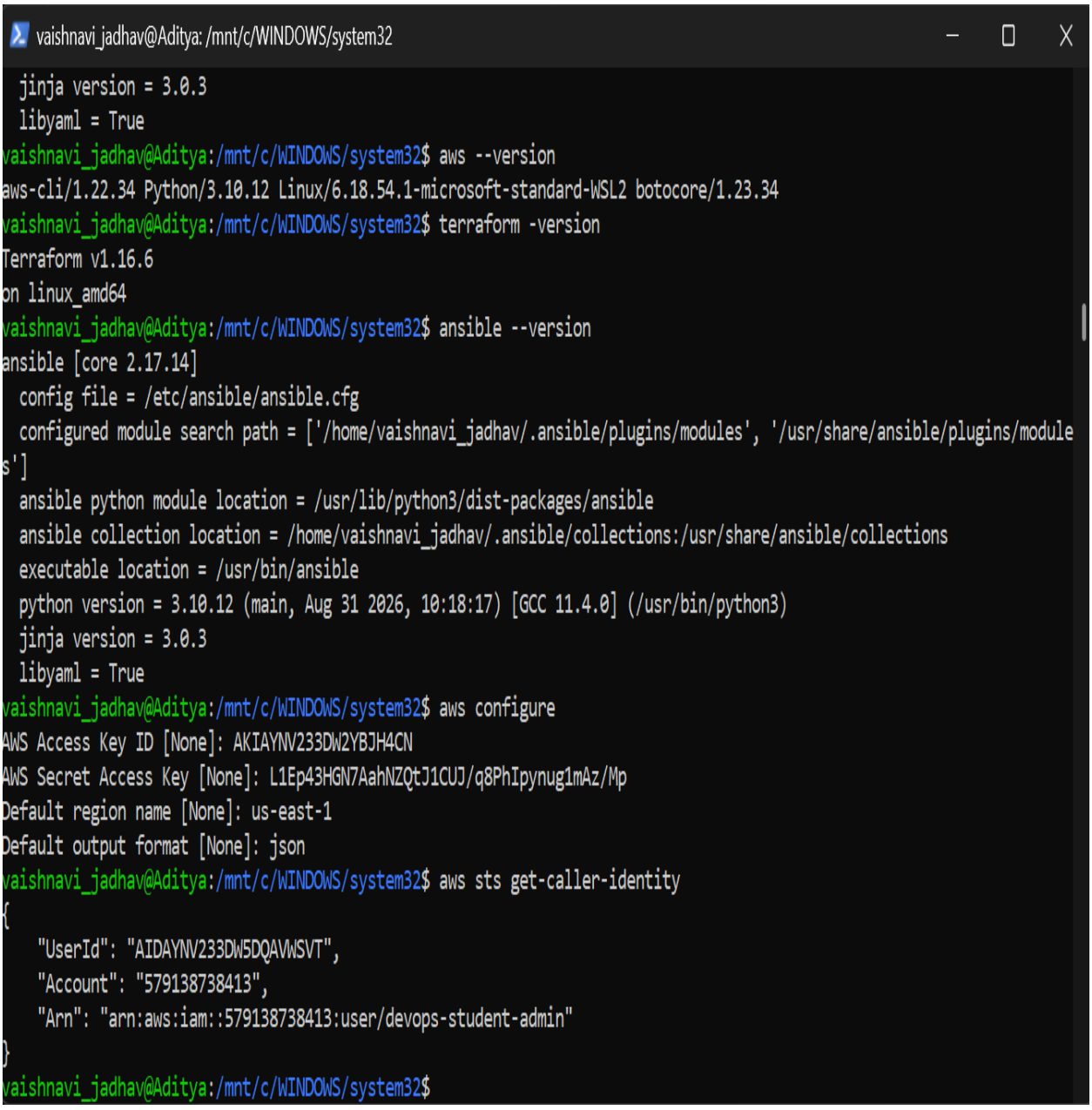
Step 1: Baseline tools, AWS CLI v2, Terraform, Ansible

- AWS CLI v2, Terraform and Ansible Core were installed on the control node. These three tools form the toolchain: Terraform builds the cloud infrastructure, Ansible configures the server, and the CLI connects both to AWS.



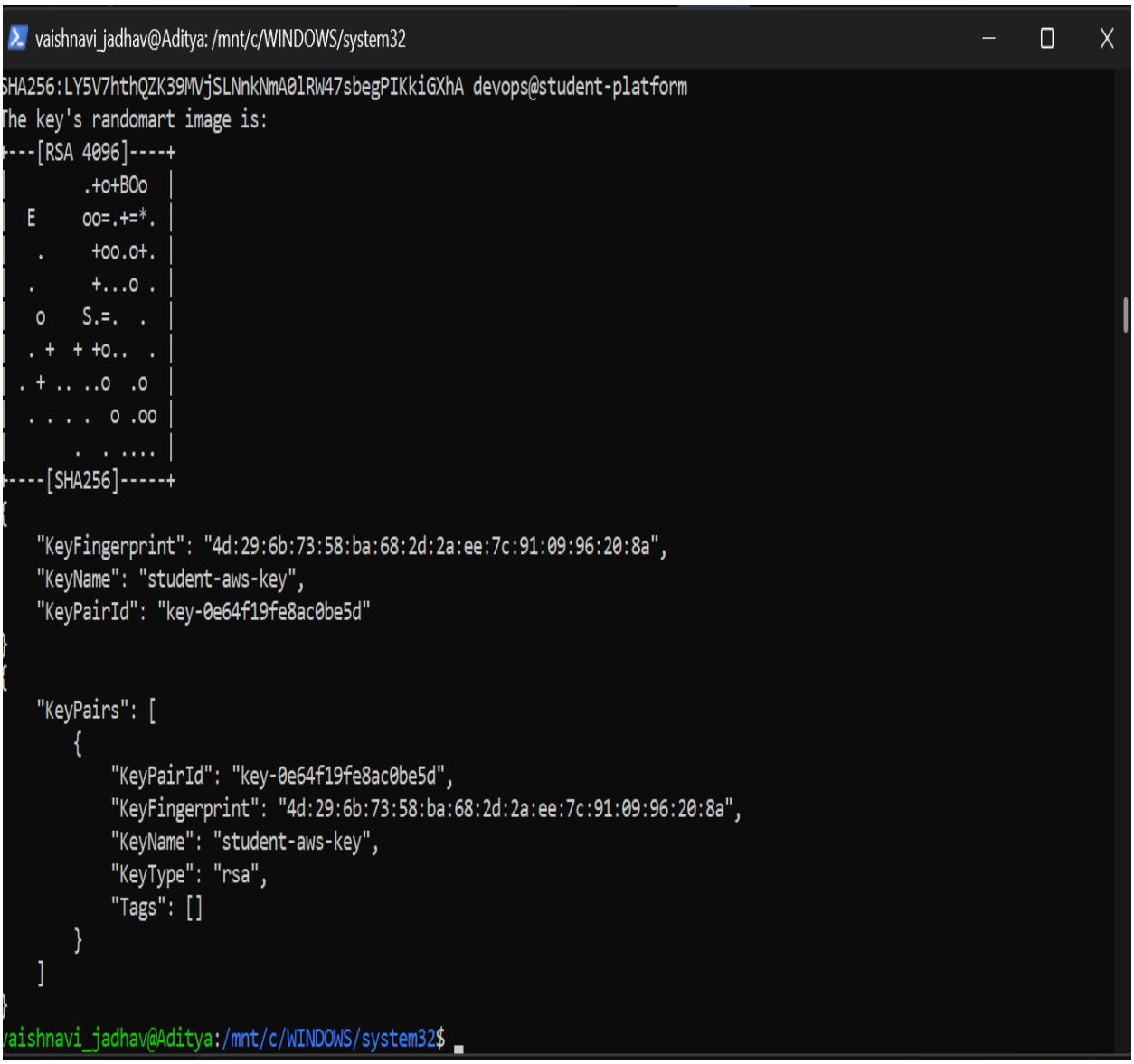
Step 2: Create an IAM user and access key

- AWS CLI authentication was configured with an IAM user. The caller-identity output confirms the credentials work, and Terraform will pick them up automatically from ~/.aws. No keys are written into the code.



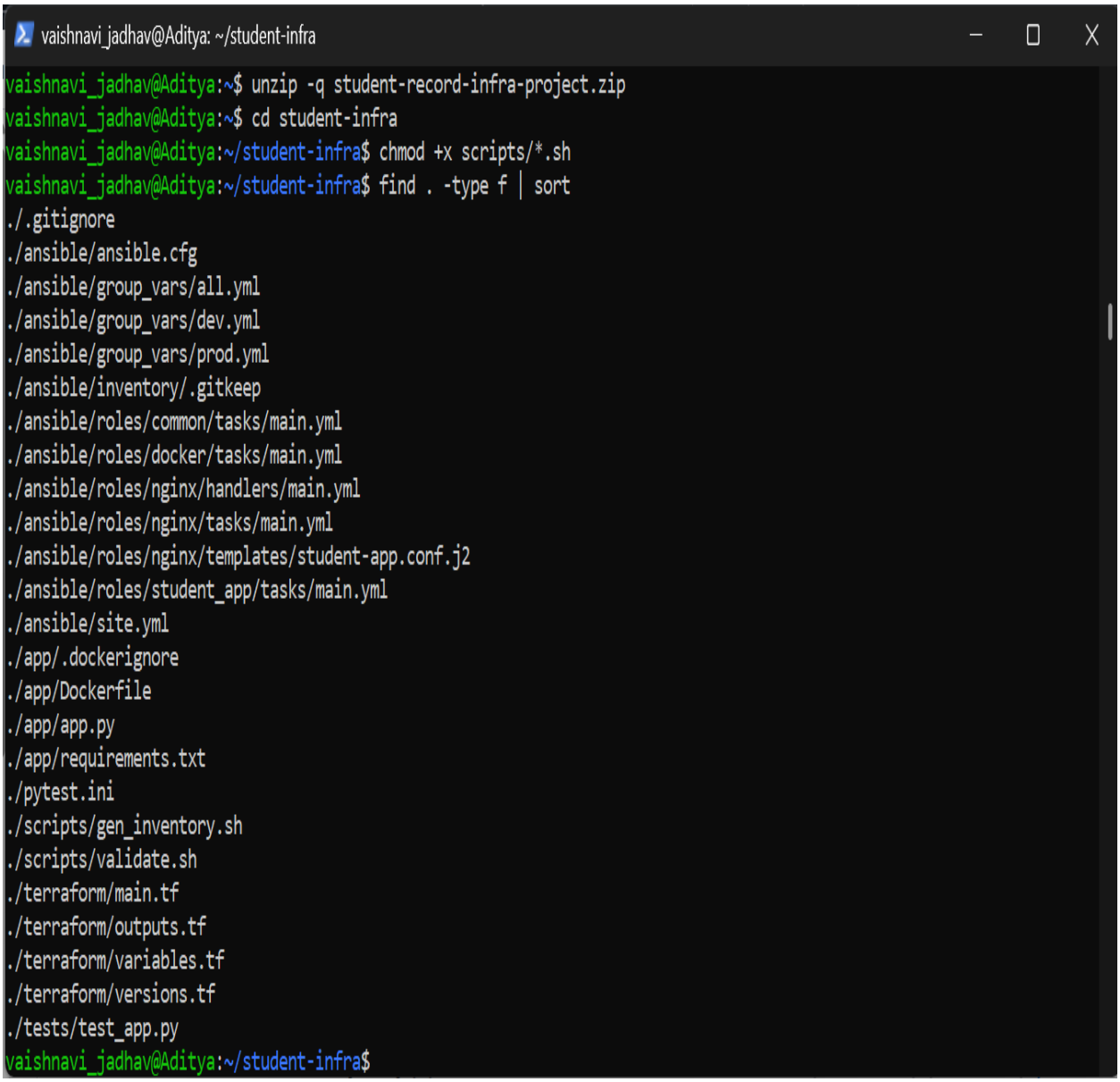
Step 3: Create and import the SSH key

- A 4096-bit RSA key pair was generated locally and only the public key was imported into AWS EC2. Ansible later uses the private key to log in to the server over SSH.



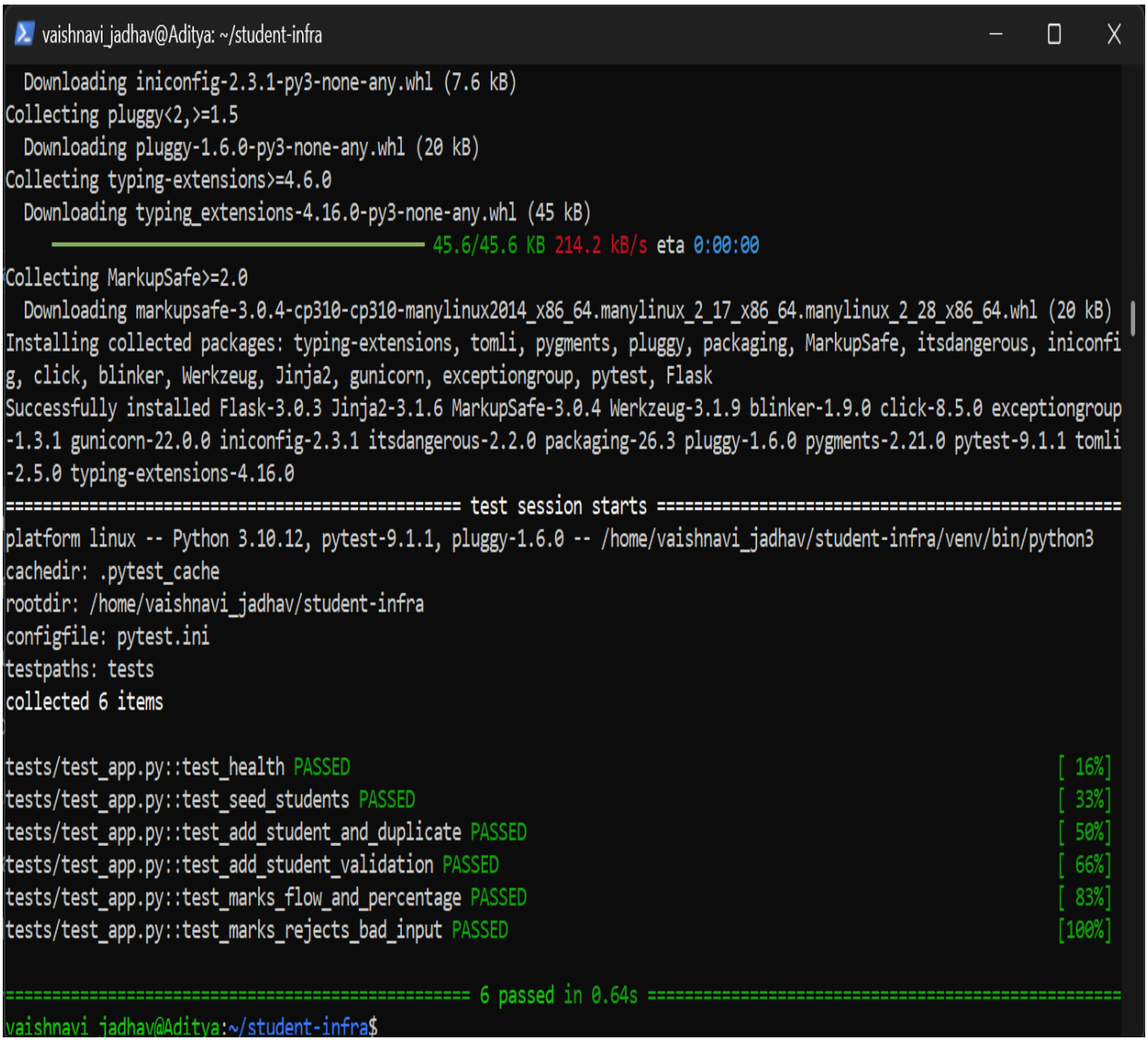
Step 4: Copy the zip into WSL

- The project follows a standard Infrastructure-as-Code layout. The terraform folder defines the AWS resources, ansible holds the roles that configure the server, app is the Flask student-record application with its Dockerfile, scripts contains the inventory and validation helpers, and tests holds the automated unit tests.



Step 5 : Run the unit tests locally

- The application's automated tests pass before deployment: health check, seeded students, duplicate and validation handling, and marks calculation. Testing before provisioning prevents deploying broken code.



Step 6: Set your IP and create a workspace

- Restricting SSH to your own IP is safer than the default `0.0.0.0/0`:

cd ~/student-infra/terraform

echo "admin_cidr = \"$(curl -s https://checkip.amazonaws.com)/32\"" > terraform.tfvars

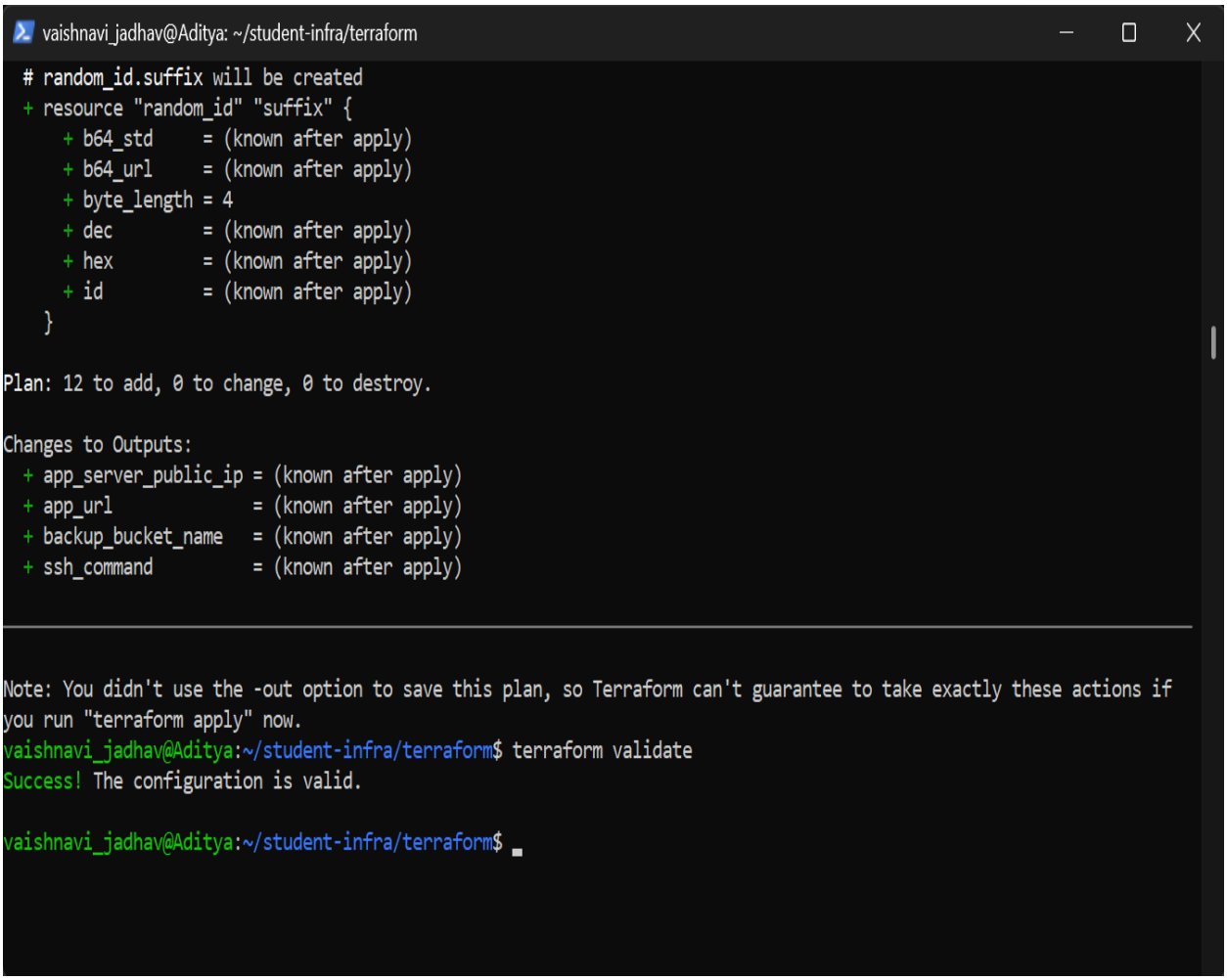
cat terraform.tfvars

terraform workspace new dev

terraform workspace show

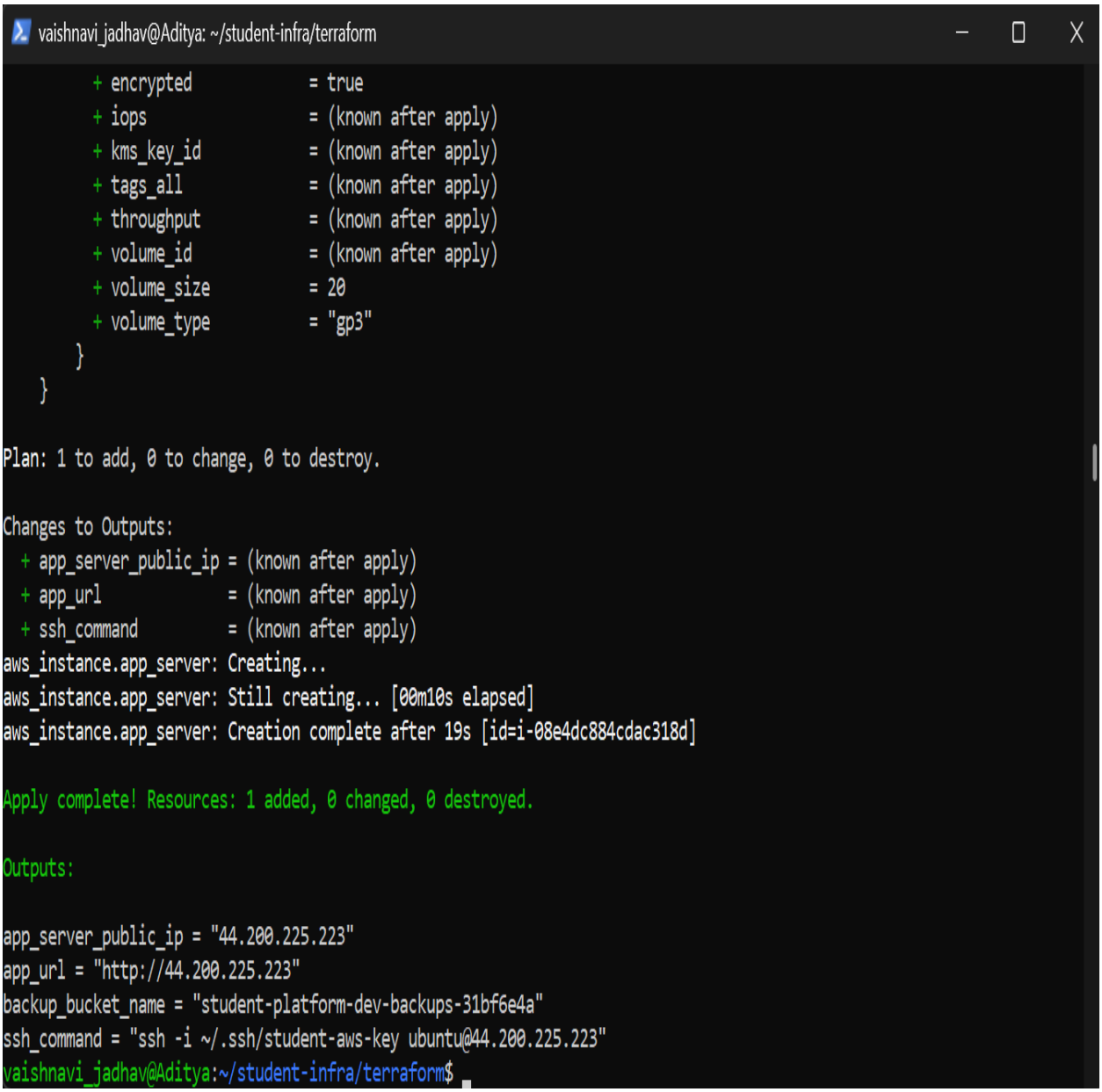
Step 7: Init, validate, plan

- Terraform downloaded the AWS and random providers, and validation confirmed the HCL syntax is correct without calling AWS. The execution plan lists 12 resources to be created: VPC, internet gateway, public subnet, route table and association, security group, EC2 instance, S3 bucket with versioning, encryption and public-access block, and a random suffix. Reviewing the plan first shows exactly what will change.



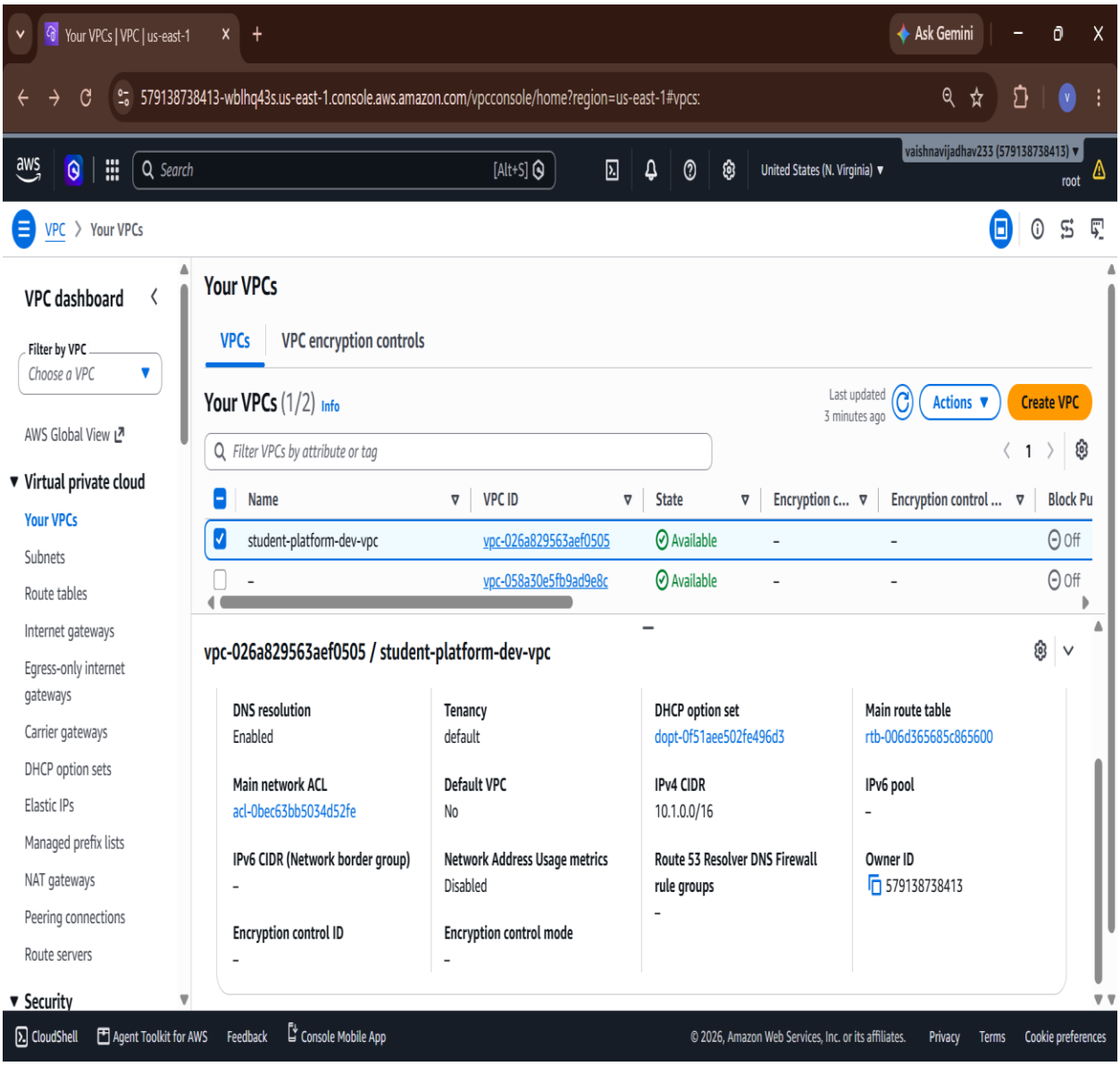
Step 8: Apply

- The AWS infrastructure was created from code. The outputs give the server's public IP, the application URL and the name of the S3 backup bucket.



Step 9: Check in the AWS Console

- The EC2 instance created by Terraform is running inside the project's VPC, with the Environment tag set to the dev workspace. The dedicated VPC (10.1.0.0/16) and the private, encrypted, versioned S3 backup bucket exist as declared in the Terraform code.



Step 10: Generate the inventory (the Terraform to Ansible handshake)

- A small script reads Terraform's output and writes the Ansible inventory automatically. This handshake passes the dynamically assigned public IP from the provisioning tool to the configuration tool, with no manual copying.

```
vaishnavi_jadhav@Aditya:~/student-infra/terraform$ cd ~/student-infra
./scripts/gen_inventory.sh
Inventory written for workspace 'dev' -> 44.200.225.223
[student_servers:children]
dev

[dev]
student-host ansible_host=44.200.225.223 ansible_user=ubuntu ansible_ssh_private_key_file=/home/vaishnavi_jadhav/.ssh/st
udent-aws-key
vaishnavi_jadhav@Aditya:~/student-infra$
```

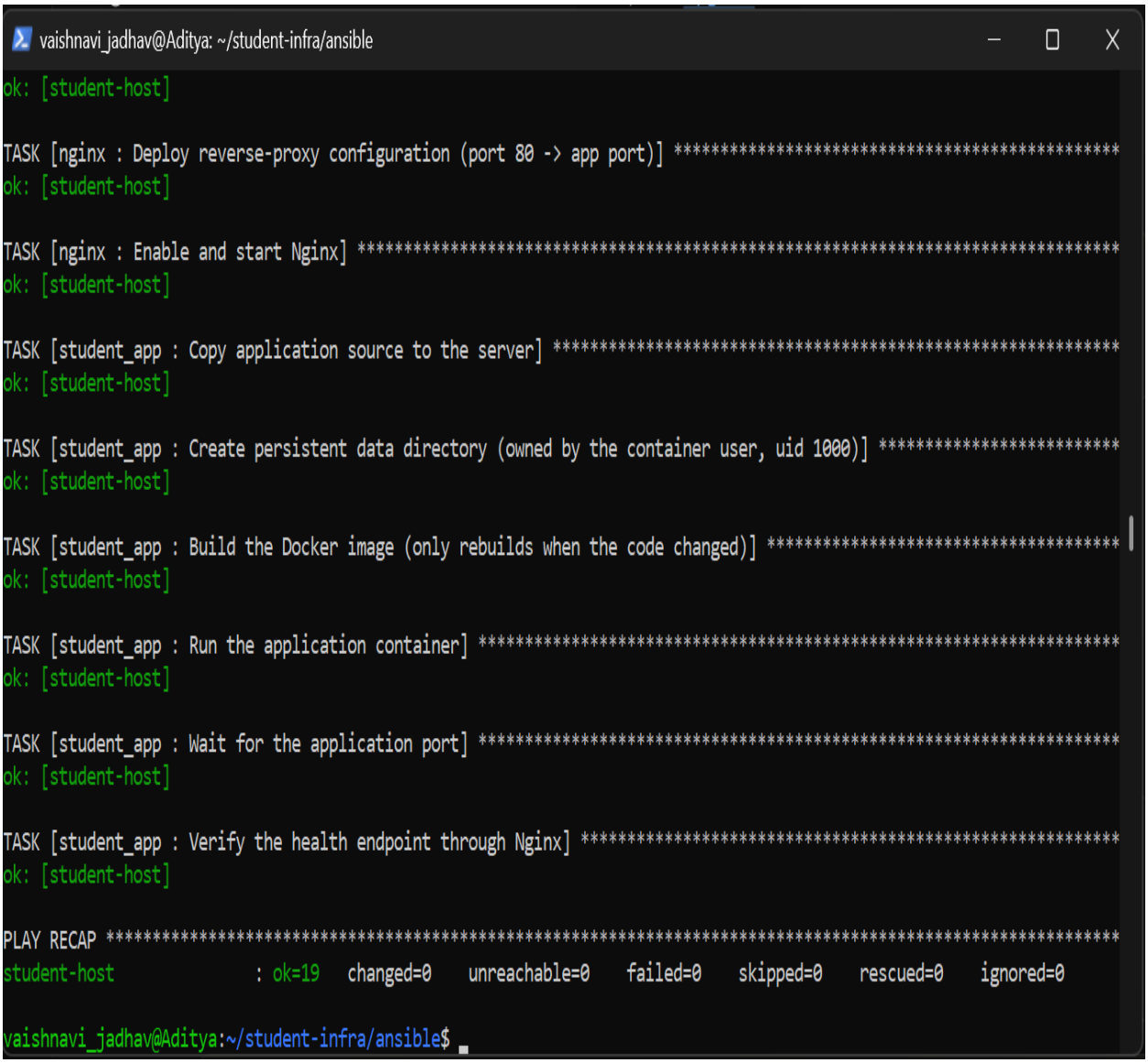
Step 11: Test the connection

- Ansible reached the new EC2 instance over SSH using the imported key. No agent is installed on the server, because Ansible only needs SSH and Python.

```
vaishnavi_jadhav@Aditya:~/student-infra/terraform$ cd ~/student-infra/ansible
ansible all -m ping
[WARNING]: Platform linux on host student-host is using the discovered Python interpreter at /usr/bin/python3.10, but
future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html for more information.
student-host | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.10"
    },
    "changed": false,
    "ping": "pong"
}
vaishnavi_jadhav@Aditya:~/student-infra/ansible$
```

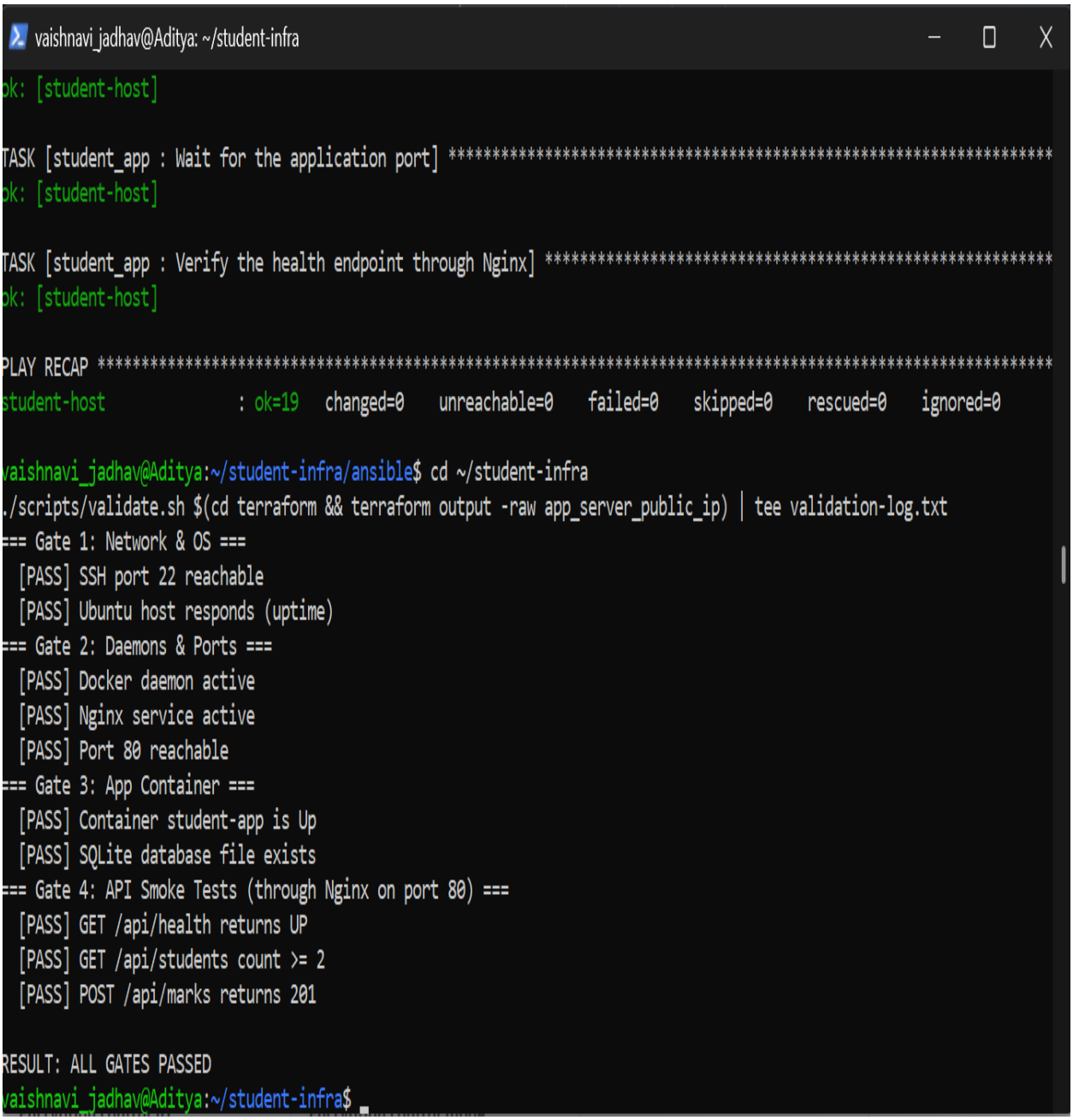
Step 12: Run the playbook

- The playbook configured the server through four modular roles: base system and firewall, Docker, Nginx reverse proxy, and the containerized student-record application. A recap with failed=0 means every task succeeded.



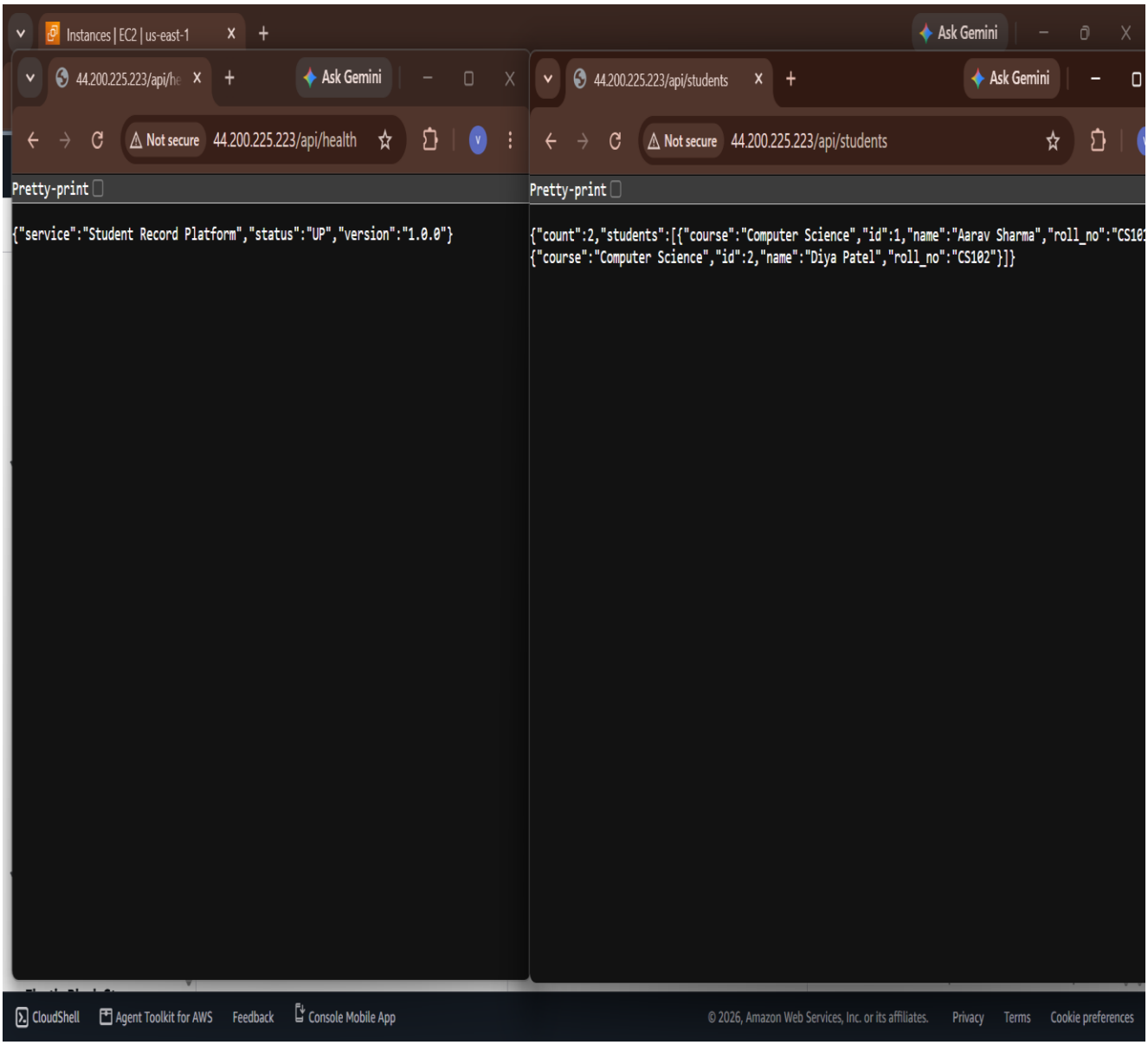
Step 13: Run the four-gate validation

- The four-gate validation passed: network and OS (SSH reachable), daemons and ports (Docker and Nginx active, port 80 open), application container (running, database file present), and API smoke tests (health returns UP, students count is at least 2, POST marks returns 201). The saved log is included as evidence.



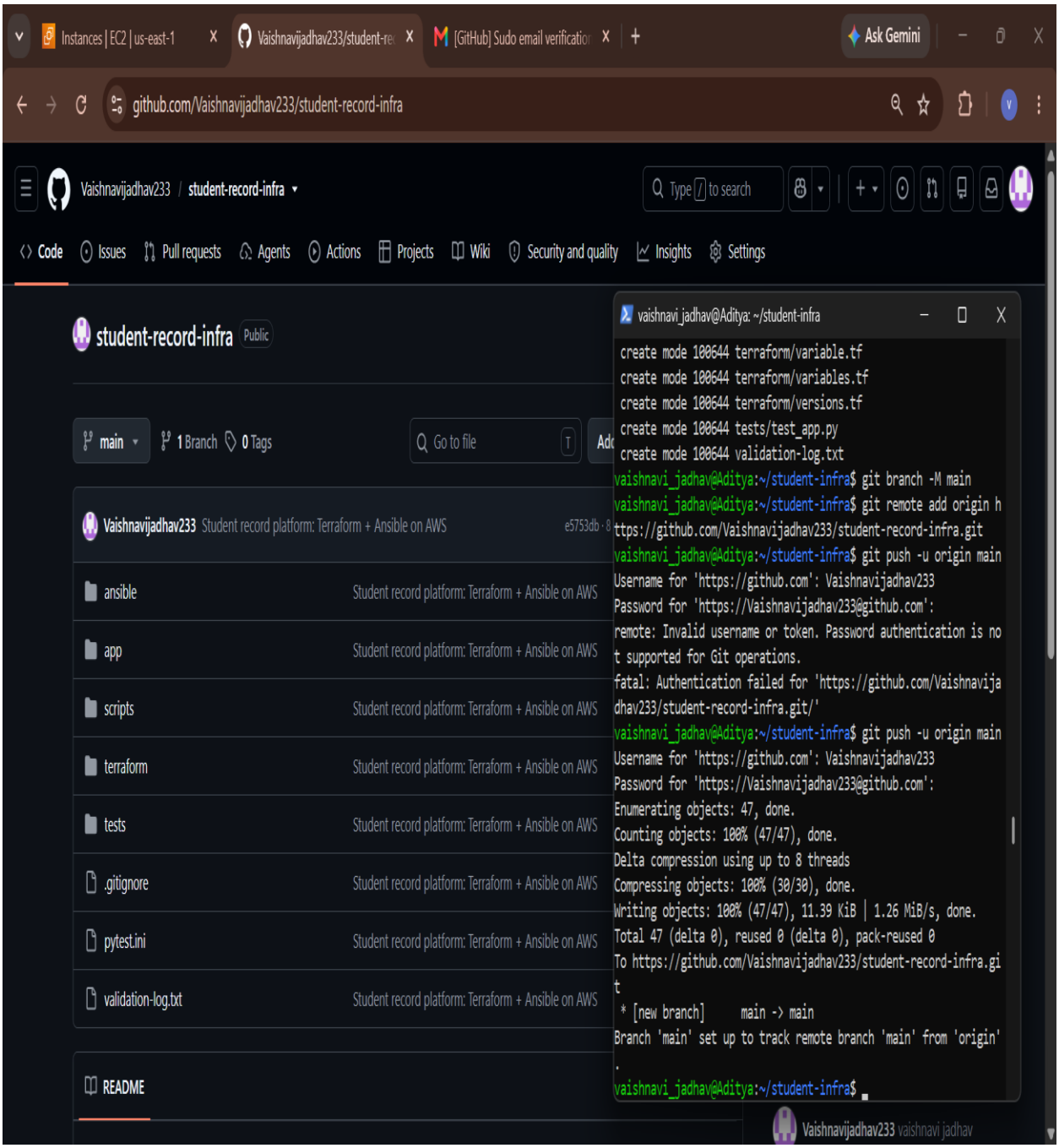
Step 14: Open the app in your browser

- The containerized Student Record application is live on AWS and reachable on port 80 through the Nginx reverse proxy. The REST API returns student records from the persistent SQLite database, confirming the application, container, and storage work end to end.



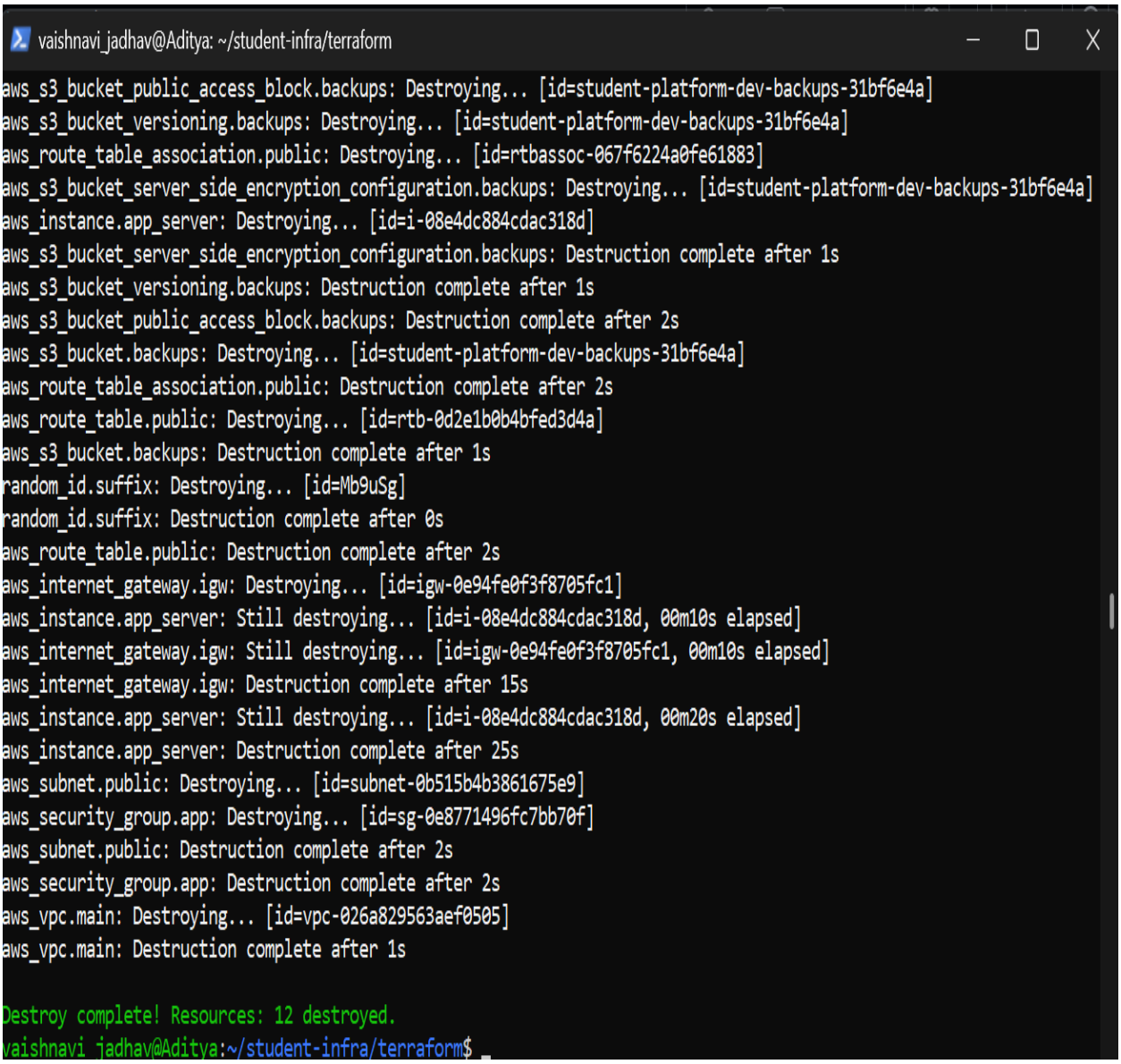
Step 15: Push to GitHub

- The project source is version-controlled on GitHub. Secrets, state files and keys are excluded through .gitignore.



Step 16: Destroy everything

- All infrastructure was removed with a single command, leaving no running resources and no ongoing charges. This is the reproducibility benefit of Infrastructure as Code.



*Github Repository Link: https://github.com/Vaishnavijadhav233/student-record-infra